

(续表)

分 类	典型产品	应用场景	优 点	缺 点
列存储	BigTable、HBase、Cassandra	分布式数据存储和管理	可扩展性强，查找速度快，复杂性低	功能局限，不支持事务的强一致性
图存储	Neo4j、OrientDB	社交网络、推荐系统，专注于构建系统图谱	支持复杂的图形算法	复杂性高，只能支持一定的数据规模

5.8.2 相关理论基础

1. CAP 理论

简单地说，CAP 理论就是对于一个分布式系统，一致性（Consistency）、可用性（Availability）和分区容忍性（Partition tolerance）三个特点最多只能三选二。

（1）一致性。一致性指系统在执行了某些操作后仍处在一个一致的状态，这点在分布式的系统中尤其明显。比如，某用户在一处对共享的数据进行了修改，那么所有有权使用这些数据的用户都可以看到这一改变。简言之，就是所有的结点在同一时刻有相同的数据。

（2）可用性。可用性指对数据的所有操作都应有成功的返回。高可用性则是在系统升级（软件或硬件）或在网络系统中的某些结点发生故障的时候，仍可以正常返回。简言之，就是任何请求不管成功或失败都有响应。

（3）分区容忍性。这一概念的前提是在网络发生故障的时候。在网络连接上，一些结点出现故障，使得原本连通的网络变成了一块一块的分区，若允许系统继续工作，那么就是分区可容忍的。

在数据库系统中，事务的 ACID 属性保证了数据库的一致性，比如银行系统中，转账就是一个事务，从原账户扣除金额，以及向目标账户添加金额，这两个数据库操作的总和构成一个完整的逻辑过程，具有原子的不可拆分特性，从而保证了整个系统中的总金额没有变化。然而，ACID 特性对于大型的分布式系统来说，可能并不总是适合，分布式系统更多的是强调可用性和分区容忍性。

由于 CAP 理论的存在，为了提高性能，出现了 ACID 的一种变种 BASE。BASE（Basic Availability, Soft-State, Eventual Consistency），即是一个弱一致性的理论，只要求最终一致性。

- Basic Availability: 基本可用。
- Soft State: 软状态，可以理解为“无连接”的，而与之相对应的 Hard State 就是“面向连接”的。
- Eventual Consistency: 最终一致性，最终整个系统（时间和系统的要求有关）看到的数据是一致的。

具体地说，如果选择了 CP（一致性和分区容忍性），那么就要考虑 ACID（传统关系数据库的基石，事务的 4 个特点）。如果选择了 AP（可用性和分区容忍性），那么就要考虑 BASE。如果选择了 CA（一致性和可用性），如 Google 的 BigTable，那么在网络发生分区的时候，将不能进行完整的操作。

ACID 和 BASE 的具体对比如表 5-14 所示。

表 5-14 ACID 和 BASE 的对比

ACID	BASE
强一致性	弱一致性
隔离性	优先考虑可用性
关注“操作提交”	尽力服务
允许事务嵌套	结果对精确度要求不高
可用性	更简单、更快速
比较“保守”	比较“激进”
难于演变	易于演变

2. 分区

现在，数据量的增加已经使得数据不可以仅在单一的计算机系统中存储（分布式的应用），尤其是为了保证数据的可靠性，有时需要复制备份。同时，为了一些规模性的操作（比如负载均衡）或者考虑到一些动态因素的影响（存储结点的改变），在设计中就要考虑“分区”的概念。

分区的一些主要方法如下：

（1）内存缓存。缓存技术可以看成一种分区。内存中的数据库系统将使用频率最高的数据复制到缓存中，加快了将数据传递给用户的速度，同时也大大减轻了数据库服务器的负担。在分布式缓存中，缓存由很多带有分配好一定内存的进程组成，它们能够放置到不同的机器上，并且可以通过配置进行应用。它的协议可以在不同的编程语言中实现，同时在用户的应用中提供了简单的键值存储 API。它通过将键值哈希散列到缓存中来存储对象。

（2）集群。数据库服务器集群在为用户提供服务时的透明性（用户感觉数据像是在同一个地方），是另外一个对数据进行分区的方法。然而，这种方法虽然能在某种程度上扩展系统数据持久层，可是集群本身的特性却仅仅只应用在了数据库管理系统的顶层，而并未在分布式最初的设计中得到应用。

（3）读写分离。指定一台或多台主服务器，所有或部分的写操作被送至此，同时再设一定数量的副本服务器用以满足读请求。如果主服务器向至少一个用户异步复制数据，这是没有写延迟的，但如果主服务器在向最后一个用户写数据还没完成的时候就崩溃了，那么写操作将是无效的；如果主服务器向用户同步复制数据，这是有延迟的，这种更新不会丢失，但读请求却不能送达副本服务器。如果对一致性要求很高的话，无法避免进一步的写延迟。在这种情况下，如果主服务器崩溃了，那么有最新的数据的副本服务器将会成为新的主服务器。这种模型（主/从模型）在读写率很高的时候工作得很好。

（4）范围分割技术/分片（Sharding）。该方法是对数据按照如下的方式进行分区操作：对数据的请求和更新在同一个结点上，并且对于分布在不同服务器上的数据存储和下载的量大致相同。从可靠性和负载均衡的观点看，数据的碎片也是需要被复制的，并且允许它们被写入主服务器的副本中和所有需要维护数据分区的副本服务器中。而为了做到这一点，需要在分区和存

储结点之间做一个映射。这个映射是动态还是静态取决于用户的应用、主服务器的“映射服务/组件”以及网络中用户应用于网络结点之间的基础结构。在分区场景中，关键在于如何将数据库中的对象映射到服务器上，通常采用哈希散列法。

3. 存储布局

存储布局确定了如何访问磁盘，以及如何直接影响性能，主要分为4种，分别是基于行的存储布局、列存储布局、带有局部性群组的列存储布局、LSM-Tree。

1) 行存储和列存储

行存储和列存储的主要区别在于，行存储将每条记录的所有字段的数据聚合存储，而列存储将所有记录中相同字段的数据聚合存储。行存储主要适用于 OLTP，或者更新操作，尤其是插入、删除操作频繁的场合；而列存储主要适用于 OLAP、数据仓库、数据挖掘等查询密集型应用。

列存储相对于行存储的优点主要有两个：

- 每个字段的数据聚集存储，在查询只需要少数几个字段的时候能大大减少读取的数据量。而查询密集型应用的特点之一就是查询一般只关心少数几个字段，而行存储每次必须读取整条记录。
- 既然一个字段的数据聚集存储，那就更容易为这种聚集存储设计更好的压缩/解压算法。

2) 带有局部性群组的列存储

带有局部性群组的列存储和列存储很相似，但是增加了局部性群组的特色。

局部性群组是在 Google 的关于 BigTable 的论文中第一次提到的。它是指根据需要将原来不存储在一起的数据，以列族为单位存储至单独的子表中。如用户对网站排名、语言等分析信息感兴趣，那么可以将这些列族放在单独的子表中，减少无用信息读取，改善存取效率。

3) LSM-Tree

LSM-Tree (Log Structured Merge Tree, 日志结构合并树) 与前面介绍的存储结构有所不同，前面介绍的存储结构在描述如何序列化逻辑数据结构，而 LSM-Tree 描述的则是为了满足高效、高性能、安全地读写的要求，如何有效地利用内存和磁盘存储。LSM-Tree 的观点是由 Patrick O'Neil 在 1996 年率先提出的。LSM-Tree 的算法思想主要用于解决日志记录索引的问题，这种应用的特点是数据量大、写速率高（2000 条/秒），又要建立有效的索引来查找日志中的特定条目。Patrick O'Neil 的做法是，在内存里维护一个相同的 B 树，当内存中的 B 树达到阈值时，批量进行滚动合并。LSM-Tree 的典型例子就是 Google 的 BigTable。

4. 查询模型

不同的 NoSQL 数据存储提供的查询功能有重大分歧。然而设计的键值存储仅提供通过主键或一些 id 字段查找的功能，并且缺乏查询更进一步域的能力，而其他数据存储（如文档数据库 CouchDB 和 MongoDB）则允许某些复杂查询。在许多 NoSQL 数据库的设计中，相对于性能和可扩展性来说，丰富的动态查询功能已被省略的现状并不奇怪。另一方面，在使用 NoSQL 数据库时，有些使用情况下需要至少有一些非主键属性查询功能。